

NobleProg

The World's Local Training Provider



Sesión 4

Comunicación, resiliencia y organización

Migración de Monolitos a Microservicios

Carlos Georg Lübbe · 3 h

Objetivo de hoy

Que el sistema resultante sea **robusto de operar** y esté **alineado con los equipos** — y cerrar con una **hoja de ruta**.

1. **Comunicación:** síncrono vs. asíncrono, eventos, gateway, contratos
2. **Resiliencia:** que el fallo parcial **no se propague**
3. **Observabilidad:** ¿dónde se rompió entre 6 servicios?
4. **Organización:** la ley de Conway y Team Topologies
5. **Síntesis:** la hoja de ruta y los errores frecuentes

1. Comunicación entre servicios

Síncrono vs. asíncrono

La decisión clave no es REST vs. RPC (como gRPC), sino **síncrono vs. asíncrono**:

- **Síncrono** (petición/respuesta): el llamante **espera** la respuesta (HTTP, gRPC)
 -  simple, resultado inmediato, fácil de depurar
 -  **acopla disponibilidad y tiempo**: si el llamado está lento, tú también; las cadenas multiplican latencia y riesgo
 -  **cuándo**: necesitas la respuesta para continuar (validar, precio, login)
- **Asíncrono**: el llamante envía y **sigue con su vida** (broker, cola)
 -  **desacopla en el tiempo**: absorbe picos, el receptor puede estar caído ahora
 -  más difícil de razonar, consistencia eventual, necesita broker
 -  **cuándo**: notificar hechos o trabajo diferido (email, PDF, propagar a otros)

La disponibilidad de una cadena síncrona es el **producto** de las disponibilidades: 5 saltos al 99,9 % $\approx$ 99,5 %.

Mensajes y eventos

Dentro de lo asíncrono, **dos intenciones distintas**:

- **Comando** (mensaje dirigido): *"haz esto"* → a un receptor concreto, por **cola** (un consumidor procesa cada mensaje). El emisor espera que algo ocurra
- **Evento**: *"esto ha ocurrido"* (`PedidoCreado`) → el emisor **anuncia un hecho** sin saber quién escucha, en un **topic** (pub/sub, N consumidores)

La **inversión de dependencia** es la gran palanca *event-driven*: facturación, fidelidad y analítica se **suscriben** a `PedidoCreado` ; el servicio de pedidos **no cambia**.

El precio: el flujo queda **implícito** (coreografía, Sesión 3) y el contrato pasa a ser el **esquema del evento** — versiónalo como una API.

El broker

La pieza central de lo asíncrono: **persiste y entrega** mensajes, gestiona suscripciones y reintentos.
(*RabbitMQ, Kafka, colas cloud — como ejemplos, no temario.*)

Conceptos que hay que conocer al elegir/operar uno:

- **Entrega al-menos-una-vez** → obliga a **idempotencia** (Sesión 3): el mismo mensaje puede llegar dos veces
- **Orden** → a menudo garantizado **solo dentro de una partición/clave**, no global
- **Dead Letter Queue (DLQ)**: a dónde van los mensajes que fallan repetidamente — **hay que monitorizarla** (si nadie la mira, ahí mueren pedidos en silencio)

Mensajería en la práctica — el productor (pedidos)

El servicio **pedidos** anuncia el hecho **PedidoCreado** en el exchange **pedidos** ; **no sabe quién escucha**:

```
// El productor declara el exchange y publica un HECHO (no conoce a sus consumidores):
channel.exchangeDeclare("pedidos", BuiltinExchangeType.FANOUT, true); // durable

var props = new AMQP.BasicProperties.Builder()
    .messageId(pedidoId)           // clave de idempotencia (dedupe aguas abajo)
    .deliveryMode(2)              // persistente: sobrevive al reinicio del broker
    .contentType("application/json")
    .build();

channel.basicPublish("pedidos", "", props, toJson(evento)); // routing key "" → fanout
```

messageId viaja con el evento (lo usa el dedupe del consumidor) · **deliveryMode(2)** = el broker lo persiste · *fanout* = cada suscriptor recibe **su copia**.

Mensajería en la práctica — el consumidor (fidelidad)

PedidoCreado → exchange pedidos ; el servicio **fidelidad** (consumidor) consume de su propia cola:

```
// The queue routes rejected messages to the DLQ (x-dead-letter-exchange):
var args = Map.of("x-dead-letter-exchange", "pedidos.dlx");
channel.queueDeclare("fidelidad", true, false, false, args);

DeliverCallback onMsg = (consumerTag, msg) -> {
    String id = msg.getProperties().getMessageId();
    long t = msg.getEnvelope().getDeliveryTag();
    if (seen(id)) { channel.basicAck(t, false); return; } // dedupe
    try {
        addPoints(parse(msg.getBody())); // business effect
        markSeen(id); // store the id in a persistent set of seen ids
        channel.basicAck(t, false); // ack → the broker deletes it
    } catch (Exception e) {
        channel.basicNack(t, false, false); // requeue=false → DLQ
    }
};
channel.basicConsume("fidelidad", false, onMsg, ct -> {}); // autoAck=false
```

`basicAck` = entrega fiable · `messageId` + `seen` = no duplicar puntos · `requeue=false` = mensaje veneno a la **DLQ**.

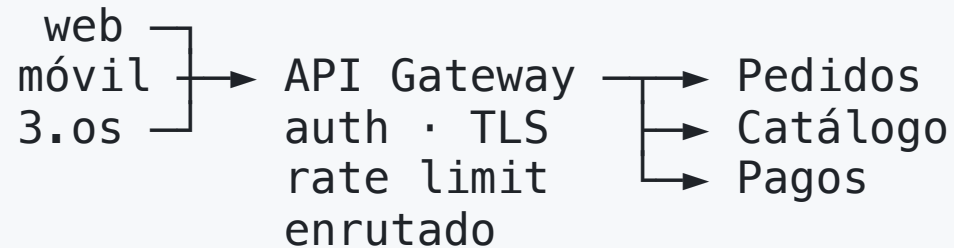
Heurística de elección

Situación	Estilo natural
Necesito la respuesta para continuar (precio, validar)	Síncrono
Notificar hechos a quien le interese	Evento (pub/sub)
Trabajo que puede hacerse después (email, PDF)	Comando por cola
Procesos de negocio entre servicios	Saga (eventos u orquestador)
Cadena síncrona de 4+ saltos	⚠ Rediseñar (copias por eventos / componer en el borde)

Síncrono en el borde (la petición del usuario quiere respuesta), **asíncrono en el interior** siempre que el negocio lo permita. Cada llamada síncrona interna evitada es disponibilidad y latencia ganadas.

API Gateway

La **fachada única** por la que entran los clientes externos (web, móvil, terceros):



- **Enrutado** al servicio interno (pieza natural del **Strangler Fig**, Sesión 2)
- **Transversales en un punto:** autenticación, rate limiting, TLS, CORS, caché
- **Adaptación al cliente:** agregar llamadas de varios servicios en una sola respuesta, traducir formatos

El gateway **adapta y protege; no decide negocio**. Riesgos: cuello de botella, "monolito de enrutado", punto único de fallo → redundante.

Descubrimiento de servicios

Con instancias que aparecen y desaparecen (escalado, despliegues, fallos): "**¿dónde está el servicio X ahora?**" necesita respuesta automática.

- **Registro de servicios:** las instancias se **registran al arrancar** y se les hace *health check*; los clientes consultan el registro (client-side) o pasan por un balanceador (server-side)
- En plataformas modernas (**Kubernetes**, como ejemplo) lo resuelve la plataforma (DNS interno + servicios) — conceptualmente lo mismo: un registro con health checks
- Clave para la migración: referenciar **por nombre lógico, nunca por IP/host** → puedes mover, escalar y **estrangular** sin tocar a los consumidores

Contratos y versionado

En el monolito el compilador vigilaba las interfaces. Entre servicios, la interfaz es un **contrato** (OpenAPI, esquemas de evento) y romperlo rompe a otros equipos **en producción**.

- **Cambios compatibles** (añadir campos opcionales): libres. *Tolerant reader*: ignora lo que no conoces
- **Cambios incompatibles** (quitar/renombrar, cambiar semántica): exigen **versionado** y convivencia (v1 y v2 en paralelo)

```
expand-contract: 1. EXPAND   añade lo nuevo junto a lo viejo
                  2. MIGRAR   consumidores pasan a su ritmo
                  3. CONTRACT retira lo viejo cuando nadie lo usa
```

Sirve para APIs, **eventos** y esquemas de BD. La independencia de despliegue se sostiene exactamente aquí.

Ejercicio 4.1

Elegir el estilo de comunicación

(ver cuaderno de ejercicios)

2. Resiliencia

El fallo es normal

En un monolito, una llamada interna no falla "a medias". En un sistema distribuido, **el fallo parcial es el estado normal**: siempre hay algo reiniciándose, una red lenta, un despliegue a medias.

No se diseña para **evitar** el fallo, sino para que **no se propague** y el sistema se **degrade con elegancia** (catálogo sin recomendaciones » nada).

Fallo en cascada: un servicio lento hace esperar a sus llamantes → agotan hilos/conexiones → tumban a *sus* llamantes → cae la plataforma.

Lento es peor que caído: un servicio caído falla rápido; uno lento **retiene recursos** de todos sus llamantes.

Timeouts, reintentos y backoff

- **Timeout:** nunca esperar indefinidamente. Ajustado a la latencia real (p99 + margen), no "30 s por si acaso" — un timeout largo **retiene recursos**
- **Reintentos:** curan fallos transitorios, pero son peligrosos:
 - Solo lo **idempotente** (¿y si el cobro sí se ejecutó y se perdió la respuesta?)
 - Solo errores **transitorios** (un 503 sí; un 400 nunca será válido)
 - **Presupuesto limitado** (2–3) y cuidado con la **amplificación en cadena:**

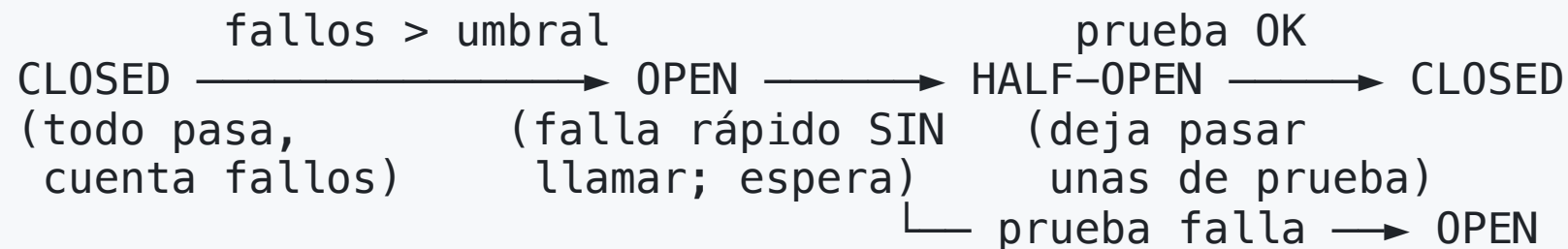
cliente → A (reintenta 3×) → B (reintenta 3×) → C
↑ recibe 3×3 = 9 peticiones

C ya está fallando — recibir 9× más carga lo hunde del todo

- **Backoff exponencial + jitter:** esperar cada vez más (1 s, 2 s, 4 s...) y con aleatoriedad — sin jitter, todos reintentan a la vez (*retry storm*)

Circuit breaker

Evita machacar a un servicio que ya falla (y protege al llamante de esperarlo):



- **Closed:** tráfico normal; cuenta fallos en silencio. Si superan el umbral → **Open**
- **Open:** falla **al instante** sin llamar a la red → el llamante no espera, el servicio enfermo respira. Tras un tiempo de espera → **Half-open**
- **Half-open:** deja pasar unas pocas llamadas de prueba. Si van bien → **Closed**; si fallan → vuelve a **Open**

Bulkheads (mamparos)

Como los **compartimentos estancos de un barco**: particionar recursos para que el agotamiento de uno no hunda el resto.

- **Pools de hilos separados por dependencia**: si recomendaciones se vuelve lento, agota *su* pool — las llamadas a pagos siguen teniendo el suyo
- **Instancias/colas separadas** por tipo de carga: el *batch* no roba capacidad al tráfico interactivo
- **Límites de concurrencia por cliente**

Sin mamparos, una sola dependencia degradada **agota recursos compartidos** y se lleva por delante funciones que ni la usaban (p. ej. el checkout cae por culpa de recomendaciones).

Ejercicio 4.2

Autopsia de un fallo en cascada

(ver cuaderno de ejercicios)

3. Observabilidad

El problema nuevo

En el monolito, un error producía **un** stack trace en **un** log. Ahora una petición atraviesa **6 servicios**: ¿dónde se rompió? ¿dónde se fue el tiempo?

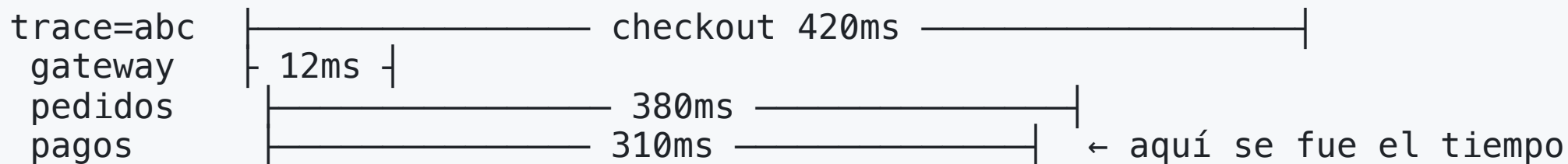
Sin inversión explícita en observabilidad, cada incidente se convierte en una **partida de Cluedo entre equipos**.

Los tres pilares:

- **Logs** — eventos discretos: *¿qué pasó?*
- **Métricas** — series numéricas agregadas: *¿cómo va el sistema?*
- **Trazas** — el viaje de una petición: *¿dónde se fue el tiempo?*

Logs, métricas y trazas

- **Logs: centralizados** (nadie hace `ssh` a 40 máquinas), **estructurados** (JSON con campos, no prosa) y con **ID de correlación** en cada línea
- **Métricas:** baratas, para **dashboards y alertas**. Señales doradas: **tráfico, errores, latencia, saturación**. Alerta sobre **síntomas del usuario** (error del checkout), no causas (CPU 80 %)
- **Trazado distribuido:** un **trace ID** se **propaga** en cada llamada; cada servicio registra sus *spans*:



Propaga el contexto **también por colas y eventos**, o la traza se corta donde empieza lo difícil.

Herramientas estándar de observabilidad

Pilar	Recoger (servicio)	Almacenar	Visualizar
Logs	OpenTelemetry	Loki, ELK, CloudWatch	Grafana
Métricas	OpenTelemetry	Prometheus, CloudWatch	Grafana
Trazas	OpenTelemetry	Jaeger, Tempo, X-Ray	Grafana

5. Aspectos organizativos

La ley de Conway

"Las organizaciones diseñan sistemas que copian su estructura de comunicación." — M. Conway, 1968

No es metáfora, es una **fuerza física** de la ingeniería:

```
3 equipos:    Frontend | Backend | DBA
               ▼ (Conway empuja)
sale esto:    UI/web   | web-api | datos    (capas, NO dominios)
```

- Migrar la arquitectura **sin tocar la organización** → monolito distribuido
- **Conway inverso**: diseña primero los equipos que *reflejan* la arquitectura deseada (equipos por **dominio**) y deja que la fuerza empuje a favor

Team Topologies

Vocabulario de Skelton & Pais para lo que una migración necesita:

- **Stream-aligned:** equipo alineado a un dominio ($\approx$ bounded context), dueño de extremo a extremo. El tipo **por defecto**; los demás existen para servirle
- **Platform:** ofrece la plataforma interna (CI/CD, observabilidad, plantillas) **como producto** autoservicio → baja la **carga cognitiva**
- **Enabling:** especialistas **temporales** que enseñan (DDD, contratos) y se retiran
- **Complicated-subsystem:** custodia una pieza de saber raro (motor de tarificación)

Carga cognitiva como límite de diseño: un equipo no debe poseer más sistema del que **cabe** en su cabeza.

Propiedad de los servicios

- **Cada servicio tiene exactamente un equipo dueño** (un equipo puede tener varios servicios; un servicio **no** puede tener dos dueños — la propiedad compartida diluye responsabilidad y calidad)
- Propiedad = ciclo de vida completo: construir, desplegar, **operar**, llevar el busca (*you build it, you run it*)
- **¿Y el monolito?** Peligro de "tierra de nadie" mientras siga sirviendo a la mayoría. Opciones: un equipo dueño con la misión de **encogerlo**, o propiedad por zonas alineada con los futuros contextos

Lo inaceptable es que el monolito **no tenga dueño**.

La migración como cambio organizativo

La migración técnica y la organizativa son **el mismo proyecto**:

- **Reorganizar** equipos hacia dominios (Conway inverso) y crear la **plataforma**
- **Formar**: operar distribuido es una habilidad nueva (on-call, debugging distribuido, diseño de contratos)
- **Gestionar la resistencia**: el experto del monolito que "pierde su feudo" necesita un papel valioso — es **quien más sabe de los dominios**
- **Patrocinio de dirección** imprescindible: una migración de años no sobrevive como proyecto de guerrilla

Ejercicio 4.4

Conway en acción

(ver cuaderno de ejercicios)

6. Síntesis y hoja de ruta

La hoja de ruta tipo

Fase 0 — Decidir y preparar (S1, S3, S4)

Objetivo + métricas con línea base + criterio de parada · bosquejo de bounded contexts · equipos hacia dominios + plataforma + CI/CD del monolito

Fase 1 — Primeras extracciones (S2, S3)

1–2 candidatos por valor×facilidad · Strangler + patrón que toque + **ACL en toda frontera** · observabilidad y contratos **desde el primer servicio** · **borrar** lo extraído · medir

Fase 2 — Industrializar

Candidatos más centrales + plantillas de servicio · **datos en serio** (propiedad, esquemas, sagas/outbox) · resiliencia por defecto

Fase 3 — Converger y parar

Genérico → comprar/SaaS · lo estable y de bajo valor **se queda** (documentado) · revisar criterio de parada → **declarar el final**

Errores frecuentes (el top 10)

1. Migrar sin **objetivo medible** ("modernizar") → sin prioridad ni parada
2. Dividir por **capas o entidades** → monolito distribuido
3. Dividir el código y **no los datos** → acoplamiento subterráneo
4. **Big-bang** de un sistema grande y vivo → objetivo móvil, cero valor
5. Cortar **demasiado fino** demasiado pronto → fusiones carísimas
6. Olvidar **borrar** el legacy → híbrido eterno, coste doble
7. Cadenas síncronas largas sin timeouts/breakers → **cascada**
8. Asumir entrega **exactamente-una-vez** → duplicados sin idempotencia
9. Observabilidad y contratos "para después" → incidentes indepurables
10. Migrar la arquitectura **sin** la organización → Conway gana

Ejercicio 4.5

Síntesis: la hoja de ruta de SaludPlus

(ejercicio final integrador — usa las 4 sesiones)

Resumen de la sesión

- **Comunicación:** síncrono solo cuando la respuesta es imprescindible; eventos para hechos, colas para trabajo diferido; gateway como fachada; contratos versionados (**expand–contract** + pruebas de contrato)
- **Resiliencia:** el fallo parcial es lo normal; timeouts, retries idempotentes con backoff+jitter, **circuit breakers** con fallback, bulkheads — que el fallo **no se propague**. *Lento es peor que caído*
- **Observabilidad:** logs estructurados centralizados, métricas sobre síntomas, trazado con propagación de contexto — **durante** la migración, no después
- **Organización: Conway gana siempre** — equipos por dominio, plataforma como producto, propiedad única, monolito nunca sin dueño
- **Hoja de ruta:** medir → extraer por valor×facilidad con ACLs → industrializar → converger y **parar**

¿Preguntas?

Fin de la formación — Migración de Monolitos a Microservicios

Decidir · Extraer con patrones · Encontrar límites y partir datos · Operar y organizar

 *¡Gracias y buenas migraciones!*